Lecture 2-11 — Context Menus & DOM Interactions

Summary

This document demonstrates a compact pattern for context menus bound to list items and a document-level fallback menu. It illustrates use of the `contextmenu` event, coordinate properties (`pageX` / `pageY`), element lookup via `closest()`, and DOM mutation (editing/removal).

Source (cleaned example)

The following snippet is the source used in the live output below.

```
<ul id="main"> ... </ul>

<div class="menu" style="display:none;">
  <div class="menu-item" id="edit">Edit</div>
  <div class="menu-item" id="delete">Delete</div>
</div>
<div class="globalmenu" style="display:none;"> ... </div>

<script>
// contextmenu handler on #main uses ev.target.closest('li')
// showMenu(menu, ev.pageX, ev.pageY) positions menu using page coordinates
// edit: prompt and set span.innerText, delete: activeli.remove()
// document click hides menus
</script>
```

Output:

- **Electricals**
  - TV
  - Fridge
  - Washing Machine

Powered by SPRK Technologies

- **Electronics**
  - Mobile
  - Laptop
    1. Lenovo
    2. Dell
    3. Samsung
  - Tablet
- **Furniture**
  - File Cabinet
  - Table
  - Chair

Operational details

Event flow

A `contextmenu` event on `#main` is intercepted; the handler uses `ev.target.closest('li')` to find the relevant list item. If an `<li>` is found, it becomes the active item for the item-specific menu. A document-level `contextmenu` provides a fallback global menu.

Positioning and coordinates

- **`pageX/pageY`** — coordinates relative to the document. Use to place menus when page may be scrolled.
- **`clientX/clientY`** — coordinates measured from the visible viewport. May differ when the page is scrolled.
- For consistent menu placement on a scrolled page prefer `pageX/pageY`.

Powered by SPRK Technologies

Element lookup: `closest()`

`closest(selector)` traverses from the event target upward through ancestors and returns the first ancestor matching the selector. It is robust when clicks originate on nested elements (e.g., a inside an <li>).

DOM mutation

Two mutations are demonstrated:

- **Edit:** obtain the item's text node (here the first element child), prompt for replacement text and set `span.innerText`.
- **Delete:** call `element.remove()` to detach the element from the DOM.

Event control and propagation

Events in the DOM travel in phases: capturing (top → target), target, and bubbling (target → top). Handlers attached without special options typically run during the bubbling phase. Two controls commonly used here are:

- `ev.preventDefault()` — prevents the browser's default behavior (for example, the native context menu).
- `ev.stopPropagation()` — prevents the event from continuing to bubble up to ancestor elements.

Why `stopPropagation()` matters in this example

In the example we have two `contextmenu` handlers: one attached to `#main` (item-level) and one on `document` (global). If an event bubbles from an item to the document, both handlers will run unless propagation is stopped.

- Without `ev.stopPropagation()` the following can happen: right-clicking an `<li>` triggers the item-level handler (showing the item menu) then the event bubbles to `document`, which also shows the global menu — resulting in conflicting UI or the item menu being immediately obscured.
- Using `ev.stopPropagation()` inside the item handler prevents the document handler from running for that event, so only the intended item menu is displayed.

Powered by SPRK Technologies

Where and when to call it

Call `ev.stopPropagation()` after you've determined that the current handler will fully handle the event (for instance, when you found a matching `<li>`). Do not overuse it — stopping propagation can prevent other legitimate listeners (analytics, keyboard handling, or delegated features) from receiving events.

Short code example:

```
main.addEventListener('contextmenu', function (ev) {
  ev.preventDefault();
  ev.stopPropagation(); // prevents document-level handler from also running
  var li = ev.target.closest('li');
  if (!li) return;
  activeli = li;
  showMenu(itemMenu, ev.pageX, ev.pageY);
});
```

Additional option: instead of stopping propagation in many places, structure handlers so that higher-level handlers first check whether the event should be ignored (for example, skip if `ev.target.closest('li')` exists). This keeps behavior explicit and minimizes surprising side effects.

ASCII: event propagation and stopPropagation

```
/* DOM tree (simplified)
window
    └ document
            └ html
                    └ body
                            └ ul#main
                                    └ li (item)
                                            └ span (event target)

Bubbling order (event travels upward from target):
span (target) → li → ul#main → body → html → document → window

Example: two handlers are present
- li handler (item menu)
- document handler (global menu)
```


Powered by SPRK Technologies

```
Without stopPropagation:
    1) Right-click on span
    2) li handler runs (shows item menu)
    3) Event continues to bubble
    4) document handler runs (shows global menu) — UI conflict

With stopPropagation in li handler:
    li handler: ev.stopPropagation();
    1) Right-click on span
    2) li handler runs (shows item menu)
    3) ev.stopPropagation() prevents further bubbling
    4) document handler does NOT run — no conflict

Code sketch:
main.addEventListener('contextmenu', function(ev){
    ev.preventDefault();
    ev.stopPropagation(); // stop here — document handler won't run
    // handle item menu
});

document.addEventListener('contextmenu', function(ev){
    // only runs when event was not stopped earlier
});
*/
```

Code (live script)

```
/* Live script controlling the example */
var itemMenu = document.querySelector('.menu');
var globalMenu = document.querySelector('.globalmenu');
var main = document.querySelector('#main');
var activeli = null;
var consoleEl = document.getElementById('console');

function hideMenus(){ itemMenu.style.display='none'; globalMenu.style.display='none'; }
function showMenu(menu,x,y){ hideMenus(); menu.style.left=x+'px'; menu.style.top=y+'px';
menu.style.display='block'; menu.setAttribute('aria-hidden','false'); }

main.addEventListener('contextmenu', function(ev){
  ev.preventDefault(); ev.stopPropagation();
```


Powered by SPRK Technologies

```
  var li = ev.target.closest('li');
  if(!li) return;
  activeli = li;
  showMenu(itemMenu, ev.pageX, ev.pageY);
  log('Item menu shown for: '+(li.innerText||li.textContent).trim());
});

document.addEventListener('contextmenu', function(ev){
  ev.preventDefault();
  activeli = null;
  showMenu(globalMenu, ev.pageX, ev.pageY);
  log('Global menu shown');
});

document.querySelector('#edit').addEventListener('click', function(){
  if(!activeli) return; var spn = activeli.querySelector('span'); if(!spn) return;
  var newText = prompt('Edit item:', spn.innerText);
  if(!newText) return; spn.innerText = newText; hideMenus(); log('Edited to: '+newText);
});

document.querySelector('#delete').addEventListener('click', function(){
  if(!activeli) return; activeli.remove(); hideMenus(); log('Item removed');
});

document.addEventListener('click', hideMenus);

function log(msg){ var t = new Date().toLocaleTimeString(); consoleEl.textContent = t+' —
'+msg; }
```